

How Far Can It Shrink?

Quantizing an English–Dhivehi Translation Model

streetsolider

Independent researcher

<https://github.com/streetsolider>

August 2026

Abstract

In earlier work we fine-tuned MADLAD-400 3B into an English→Dhivehi translation model that reaches 61.9 chrF++ on a frozen 1,500-pair news test set. To run it, you need about 6 GB of GPU memory and a working PyTorch environment, and not everyone has that. In this paper we try to make the same model run on ordinary computers by quantizing it, and we measure what each step of compression costs in translation quality. We merge the LoRA adapter into the base weights and quantize the merged model in two ways: with bitsandbytes, which stays inside Python on a GPU, and to GGUF k-quants for llama.cpp, which runs on a plain CPU with no Python at all. Down to 4 bits, nothing is lost. The Q4_K_M file is 1.86 GB ($3.2\times$ smaller than the bfloat16 original), scores 60.91 chrF++ against the original’s 60.88 under the same decoding, and translates about one news sentence per second on eight CPU threads. Below 4 bits quality does fall: by 0.79 chrF++ at 3 bits and 3.58 at 2 bits. Two results surprised us. First, the two 256k-vocabulary tensors, which we expected to be the fragile part of a Thaana model, survive 2-bit quantization cheaply (0.25 chrF++ for the embedding, 0.88 for the output projection), while removing a similar number of megabytes from the encoder–decoder body costs 2.79. The output projection is about $3.5\times$ more sensitive than the embedding even though the two tensors have the same shape and size. Second, the script-validity check from our earlier work, a round trip through the Segha keymap, turns out to be blind to quantization damage: quantized models fail by repeating valid Thaana words or by inserting foreign characters that the keymap passes through, so the check stays above 99.9% even at 2 bits. We release the quantized models and the evaluation code.

1 Introduction

In earlier work we fine-tuned MADLAD-400 3B-MT [8] with LoRA [7] into an English→Dhivehi translation model that reaches 61.8 chrF++ on a frozen 1,500-pair news test set. That model works, but it is not easy to run. In bfloat16 it needs roughly 6 GB of weights plus working memory, which in practice means a dedicated GPU and a Python environment with PyTorch, CUDA and `transformers` installed. Not everyone in the Maldives who wants to translate a paragraph has a machine like that, and a model that only runs on the computer that trained it is not doing much for the language community it was built for.

So this paper asks a narrow, practical question: how far can this model be compressed before its Dhivehi output stops being good? We take the fine-tuned model, merge the adapter into the base weights, and walk the result down two quantization paths. The first is bitsandbytes [2, 3], which quantizes the model inside PyTorch for GPU inference. The second is GGUF k-quants [5, 6], which produce a single self-contained file that llama.cpp can run on a CPU, a GPU, or both. At every step we measure translation quality, script validity, file size, memory and speed.

Dhivehi makes this more interesting than a routine compression exercise. Thaana is a right-to-left script that no other language uses, and MADLAD’s 256k SentencePiece vocabulary gives the model two tensors of $256,000 \times 1024$ parameters (the token embedding and the output projection) that together hold 524M of its 2.94B parameters, or 17.8%. These two tensors are the part of the model that decides which Thaana characters come out. The two toolchains also treat them very differently: bitsandbytes leaves them in 16-bit, which puts a 1.05 GB floor under any 4-bit bitsandbytes build, while GGUF quantizes them along with everything else. We test what that difference costs with per-tensor probes that protect or damage the vocabulary tensors independently of the rest of the network.

We also carry over the script check from the ear-

lier work: an output passes if it survives a round trip through the Segha keymap [12], a one-to-one mapping between Thaana characters and ASCII. We expected this check to give us an early warning of quantization damage, independent of chrF++. As we will show, it did not, and the reason why is one of the more useful things we learned.

2 Background

2.1 Weight-only post-training quantization

Both methods we evaluate are weight-only post-training quantization: the trained weights are rounded to a lower-precision format and converted back on the fly during matrix multiplication, while activations stay in 16-bit. Nothing is retrained and no calibration data is needed, which is what makes these methods cheap enough to apply to a finished model.

bitsandbytes provides NF4 and FP4, two 4-bit formats introduced with QLoRA [3], and the 8-bit LLM.int8() scheme [2]. NF4 is designed around the fact that trained weights are roughly normally distributed, and double quantization compresses the per-block scales further. These formats run inside PyTorch, so the model stays a normal **transformers** object and beam search keeps working.

GGUF k-quants [5] are the storage format used by llama.cpp [6]. Weights are stored in blocks with hierarchical scales, at nominal rates from 8 bits (Q8_0) down to about 2.6 bits (Q2_K), and mixed variants such as Q4_K_M keep a few sensitive tensors at higher precision. The resulting file carries its own tokenizer and runs with no Python at all, which is what makes this the format that matters for our goal of running on ordinary computers.

We do not evaluate GPTQ [4] or AWQ [9]. Both are calibration-based methods built for decoder-only LLMs, and neither toolchain currently supports T5-style encoder-decoder models like MADLAD. Making them work on a seq2seq model with a 256k vocabulary would be a project of its own.

2.2 What we expected quantization to cost

Studies of weight-only quantization find 4-bit precision to be close to the best accuracy per bit for large language models [1], with quality dropping sharply below that. It was not obvious to us that this would carry over to our setting, for two reasons. First, our model is small by current standards (3B), and quantization error tends to hurt smaller models more. Second, it is a fine-tuned encoder-decoder doing one narrow task in

a low-resource script, not a general-purpose LLM, and what we care about is translation quality in Thaana rather than perplexity in English. The earlier work gave us one relevant data point: the untuned 3B base model lost only 0.3 chrF++ under NF4, while the 7B backtranslation checkpoint became unusable under the same settings and fell into repetition loops. Neither tells us how the fine-tuned model behaves, which is what we measure here.

3 Method

3.1 Merging the adapter

Every quantization path needs a single set of weights, so the first step is to merge the LoRA adapter into the base model. This is also where a known problem with the MADLAD checkpoint has to be fixed once and for all. The published checkpoint stores the true shared embedding as `decoder.embed_tokens.weight` and a separate, untied `lm_head.weight`, but its configuration says `tie_word_embeddings=True`. Recent versions of **transformers** resolve that contradiction by tying all three tensors to the wrong one, which destroys generation completely.

Our loader repairs this at load time, but a runtime repair does not help a converter that reads the files from disk. So we save the merged model with `tie_word_embeddings=False` and both tensors written out, and then verify on the saved files (reloaded with plain **transformers**, and again after GGUF conversion) that the output projection is still a different tensor from the token embedding. In the converted GGUF this shows up as `output.weight` and `token_embd.weight` holding different values.

3.2 Decoding parity

The two tracks cannot use the same decoding algorithm, because llama.cpp has no beam search for encoder-decoder models. Every GGUF number in this paper is therefore greedy, while the bitsandbytes track keeps the beam-4 decoding used in the earlier paper. Comparing a greedy quantized model against a beam-4 baseline would blame quantization for the loss that actually comes from dropping beam search, so we measured two baselines from the same merged bf16 weights: beam 4 (61.86 chrF++) and greedy (60.88). Beam search is worth 0.98 chrF++ here. GGUF rows are always compared against the greedy baseline, bitsandbytes rows against the beam-4 baseline, and we never compare across the two.

Configuration	Size	chrF++	Δ	RT
<i>transformers, beam 4</i>				
bf16 (merged)	5.88 GB	61.86	—	100%
int8	4.00 GB	61.76	−0.10	100%
NF4	3.06 GB	61.93	+0.07	100%
<i>llama.cpp, greedy</i>				
bf16 (merged)	5.88 GB	60.88	—	100%
Q8_0	3.13 GB	60.88	0.00	100%
Q6_K	2.42 GB	61.02	+0.14	100%
Q4_K_M	1.86 GB	60.91	+0.03	100%
Q3_K_M	1.47 GB	60.09	−0.79	99.9%
Q2_K	1.18 GB	57.30	−3.58	100%
Q2_K, vocab 8-bit	1.37 GB	57.57	−3.31	100%
<i>per-tensor probes, vs. Q6_K</i>				
emb. at 2-bit	2.29 GB	60.77	−0.25	100%
out. at 2-bit	2.29 GB	60.14	−0.88	100%

Table 1: Full frozen devtest (1,500 pairs). Δ is chrF++ against the baseline decoded the same way: beam-4 rows against bf16 beam 4, greedy rows against bf16 greedy. The two probes are measured against Q6_K, since that is the setting they modify. RT = Segha keymap round-trip pass rate. Sizes are decimal GB.

3.3 Experiment protocol

We evaluate on the same frozen 1,500-pair news devtest set as the earlier work, with the same metrics: chrF++ [10], character-level BLEU via sacreBLEU [11], and the Segha keymap round-trip pass rate. A full GGUF run costs about an hour per artifact, so we used a two-stage protocol: every artifact was first screened on the first 150 sentences, and the informative points (the extremes, the standard 4-bit setting, and the region where quality starts to move) were then run on the full 1,500. Screened numbers are always compared against the baseline rescored on the same 150 sentences, never against its full-split score.

Speed and memory are measured separately on a fixed, seeded 64-sentence subset, because our evaluation harness starts one llama.cpp process per sentence and would otherwise count a model load against every sentence. For llama.cpp we report the decode rate from its own timing counter, which excludes model loading, and we report GPU and CPU-only configurations separately. All measurements come from one machine, an RTX 5070 Ti (16 GB) with a 16-thread CPU and 31 GB of RAM, running Windows 11.

<i>Source:</i>	Maldivian introduces premium economy tickets for domestic flights					
Reference						
bf16	to					
Q4_K_M						
Q3_K_M						
Q2_K		CJK				

Table 2: One devtest sentence down the ladder. Every level from bf16 greedy to Q4_K_M produces the identical, correct output. Q3_K_M mistranslates “premium” as مؤید (“primary”). Q2_K drops the word for “domestic” and instead emits the *Chinese* word for it, two CJK characters shown here as a box. That output still passes the keymap round-trip check, because the keymap only rewrites Thaana and ASCII and passes other characters through untouched.

4 Results

4.1 Quantization is free down to 4 bits

Table 1 contains the practical result of this paper. Neither bitsandbytes format costs measurable quality: NF4 actually lands slightly above the bf16 baseline (61.93 vs. 61.86) and int8 a tenth of a point below it. Both differences are far smaller than the gap between the two decoding strategies. On the llama.cpp side the picture is the same until 4 bits. Q8_0 reproduces the bf16 greedy baseline almost exactly; on a 20-sentence parity check, 19 of 20 outputs were byte-identical to the PyTorch model, which is our strongest evidence that the conversion and the untying fix are correct. Q4_K_M, at 1.86 GB, is still within noise of the baseline.

That is a 3.2 $\times$ reduction from the 5.88 GB bf16 checkpoint with nothing given up, and it is the point where the model stops needing a GPU (Section 5).

4.2 Where it breaks, and how

Below 4 bits the curve turns (Figure 1, Table 3). On the full test set, Q3_K_M gives up 0.79 chrF++ and Q2_K gives up 3.58. Even Q2_K is not useless: at 1.18 GB it still scores above every earlier system we benchmarked in the previous paper, including the mT5-large fine-tune at 55.75. But the losses are now real.

The interesting part is how it breaks. Table 2 follows one devtest sentence down the ladder. From bf16 all the way to Q4_K_M the output is byte-identical. At Q3_K_M the sentence is still fluent but one word drifts: “premium” becomes “primary”. At Q2_K the model inserts two Chinese characters in place of a Dhivehi word. This is typical of what we saw when reading outputs: quality does not collapse, it erodes word by word. Counting outputs that stay byte-identical to bf16 greedy on the 150-sentence screen gives a smooth ordering: 111 of 150 at Q5_K_M, 69 at Q3_K_M, 25 at Q2_K.

The worst failures at Q2_K are repetition loops. Two of the 150 screened outputs run past twice the reference length, one of them repeating 40 times where the reference is a six-word headline, and those few sentences account for a large share of the corpus-level drop (the mean output/reference length ratio is only 1.025). This is the same way the quantized 7B checkpoint failed in our earlier work.

What we did not expect is that our script-validity check would miss all of this. Across the full test set the keymap round-trip pass rate never falls below 99.93%. Q2_K passes all 1,500 sentences, and the single failure in the entire study is one sentence at Q3_K_M, a model that is otherwise better than Q2_K by 2.8 chrF++. So the signal is not just weak, it is not even ordered the same way as quality. Looking at the failure modes, the reason is clear. A repetition loop is made of perfectly valid Thaana words, so it round-trips cleanly. And the Chinese characters in Table 2 also pass, because the keymap rewrites only Thaana and ASCII and passes everything else through. The check was designed to catch a model writing the wrong script, and these models fail differently: they write the right script with the wrong content. Corpus chrF++ and a length-ratio check are what actually detect this.

4.3 The vocabulary tensors are not the fragile part

Going in, we expected the two $256,000 \times 1024$ vocabulary tensors to be where a Thaana model breaks first. They hold 17.8% of the parameters, and they are the layers that map between token identities and the representation space, so quantizing them coarsely should blur exactly the distinctions between Thaana tokens. The probes say otherwise, though not in a symmetric way.

Dropping the token embedding alone from 6 bits to 2, with the rest of the model kept at Q6_K, costs only **0.25** chrF++. Doing the same to the output projection costs **0.88**. Both are small next to what the body costs: moving the body from Q3_K_M to Q2_K gives up 2.79 for a comparable 298 MB. So the main conclusion is that the vocabulary is not where

Artifact	Size	chrF++	Δ
bf16 greedy (slice)	5.88 GB	62.99	—
Q8_0	3.13 GB	63.22	+0.23
Q6_K	2.42 GB	63.50	+0.51
Q5_K_M	2.13 GB	63.10	+0.11
Q4_K_M	1.86 GB	63.54	+0.55
Q3_K_M	1.47 GB	61.76	-1.23
Q2_K	1.18 GB	58.98	-4.01
<i>per-tensor probes</i>			
Q4_K_M, vocab 8-bit	2.05 GB	63.28	+0.29
Q2_K, vocab 8/6-bit	1.37 GB	60.22	-2.77
Q6_K, emb. 2-bit	2.29 GB	63.15	+0.16
Q6_K, out. 2-bit	2.29 GB	63.26	+0.27

Table 3: Screening sweep, first 150 devtest sentences, greedy decoding. The baseline is rescored on the same 150 sentences. Every artifact passed the keymap round trip on all 150. As Section 4.3 shows, several of these screen deltas did not survive the full run.

this model is fragile. But the two tensors are clearly not interchangeable either. They have identical shape and size and sit at opposite ends of the same network, yet one is about $3.5\times$ more sensitive than the other.

We think the asymmetry has a simple explanation. The embedding is only used to look up a vector for each token, and a lookup that lands slightly off still passes through 32 layers of computation that can absorb the error. The output projection produces the logits directly: its errors are the last thing that happens before the argmax picks a token, with nothing after them to compensate. If that explanation is right, the ordering should hold for other encoder-decoder models too, and the practical advice is that llama.cpp’s `--output-tensor-type` option deserves more care than `--token-embedding-type`.

Consistent with all this, protecting both vocabulary tensors in an otherwise 2-bit model (embedding at 8-bit, output projection at 6-bit, costing 192 MB) recovers only 0.27 of the 3.58 chrF++ lost. Spending those megabytes on the body instead buys roughly ten times as much. The rule that falls out is simple: spend bits on the body.

We also want to be honest about something that almost misled us. On the 150-sentence screen, these two probes measured -0.35 and -0.24 , which put the embedding on the *wrong* side of the comparison and made both look like noise. The full test set separates them cleanly and reverses the ordering. Screening was still the right way to spend our compute, because it correctly picked which artifacts deserved a full run. But on differences under one chrF++ point, a 150-sentence screen resolved neither the size nor even the sign of the effect, and we would not have been comfortable publishing either number from the screen alone.

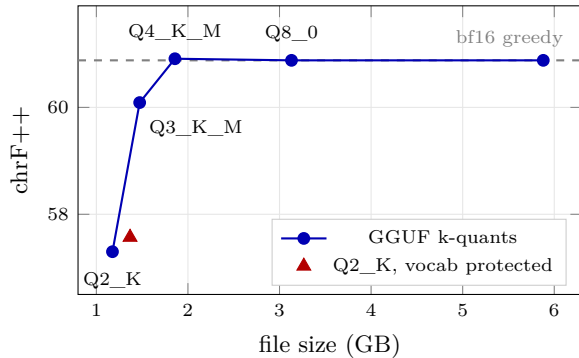


Figure 1: Quality against file size on the full 1,500-pair devtest, greedy decoding. The chrF++ axis is zoomed to the 56.5–62 range so the small differences are visible. The curve is flat from 5.88 GB down to 1.86 GB and only then turns. Protecting the vocabulary tensors at 2 bits (triangle) buys back 0.27 chrF++ for 192 MB; spending a similar amount on the body instead (Q2_K to Q3_K_M, 298 MB) buys back 2.79.

5 Speed and memory

Two patterns in Table 4 matter in practice.

On a GPU, quantization buys memory, not speed. Decode rates across the whole GGUF ladder sit in a narrow band of 163 to 184 tokens/s, even though the files differ by a factor of 2.7. At this model size the decode loop is not limited by how fast weights can be read, so smaller weights do not make it faster. What changes is the footprint: the whole llama.cpp process peaks at 2.79 GB at Q4_K_M, against 8.82 GB for bf16 beam-4 generation in PyTorch.

bitsandbytes helps peak memory much less than its weight compression suggests. NF4 stores the weights in 3.06 GB, but peak allocation only falls from 8.82 to 6.03 GB, because the unquantized vocabulary tensors, the activations and the beam-search state are all still there. int8 was the slowest configuration we measured at 32 tokens/s, a third of plain bf16, which is a known cost of LLM.int8()’s mixed-precision scheme. For this model, int8 loses on every axis: it is larger than NF4, slower than everything else, and no more accurate.

On a CPU, quantization does buy speed. Here the rates follow bit-width directly, from 17.6 tokens/s at Q8_0 to 29.1 at Q2_K, because CPU decoding is limited by memory bandwidth in exactly the way GPU decoding is not. At Q4_K_M the model does 22.7 tokens/s on eight CPU threads, which is roughly one news sentence per second, from a 1.86 GB file, in under 3 GB of RAM, with no GPU involved. That is the build we would hand to someone who wants to run this model on an ordinary laptop, and it is the one we recommend.

Configuration	Peak mem.	tokens/s	
		GPU	CPU
tf. bf16, beam 4	8.82 GB*	108	—
tf. bf16, greedy	6.63 GB*	148	—
tf. NF4, beam 4	6.03 GB*	76	—
tf. int8, beam 4	6.95 GB*	32	—
GGUF Q8_0	3.93 GB	163	17.6
GGUF Q6_K	3.28 GB	169	19.9
GGUF Q4_K_M	2.79 GB	178	22.7
GGUF Q3_K_M	2.44 GB	168	24.9
GGUF Q2_K	2.17 GB	184	29.1

Table 4: Throughput and peak memory on a 64-sentence subset. tf. = transformers. *transformers peak memory is `torch.cuda.max_memory_allocated`, which excludes the CUDA context; GGUF peak memory is sampled from `nvidia-smi` and includes it, so the GGUF column is if anything pessimistic. transformers rates are totals over batches of 16; llama.cpp decodes one stream. CPU is 8 threads with no GPU offload.

6 Discussion and limitations

Our results are for one model, one language direction, and one test domain (news). We would guess that the shape of the curve (flat to 4 bits, a knee below it) is typical for a fine-tuned 3B encoder–decoder, but we cannot generalize the exact knee position from a single system, and a model fine-tuned on a narrower task might break earlier.

The two tracks are not measured under identical decoding, because llama.cpp does not implement beam search for encoder–decoder models. We handled this by measuring both baselines and comparing like with like, but it means every GGUF number is a greedy number: a Q4_K_M deployment gives up the 0.98 chrF++ that beam search contributes, on top of the negligible quantization loss. An encoder–decoder beam search in llama.cpp is the single change that would most improve the deployed model.

We did not attempt quantization-aware training, distillation to a smaller student model, or pruning. All three are plausible routes to something smaller than 1.86 GB without the 3-bit quality loss, and all three require training, which post-training quantization deliberately avoids. Since our setting is calibration-free, we also never used the training data at quantization time; methods like GPTQ and AWQ that do use calibration data might push the knee lower, if they gain encoder–decoder support.

Finally, all measurements come from one Windows machine with one GPU. The quality numbers do not depend on hardware, but the throughput numbers in Section 5 should be read as a single data point, not a

benchmark.

7 Conclusion

A 3B English→Dhivehi translation model that scored 61.8 chrF++ in bfloat16 can be shipped as a 1.86 GB file that scores the same, runs from a single binary with no Python, and does not need a GPU. Below 4 bits quantization costs real quality (0.79 chrF++ at 3 bits, 3.58 at 2 bits), and it fails by repeating valid Thaana or inserting foreign characters rather than by breaking the script, which means the script-validity check that serves this language pair well elsewhere cannot detect it. The 256k-token vocabulary tensors, which we expected to be the sensitive part of a Thaana model, tolerate 2-bit quantization cheaply, though not equally: the output projection is about 3.5× more sensitive than the token embedding despite being the same size, so `--output-tensor-type` deserves more care than `--token-embedding-type` when quantizing an encoder-decoder aggressively. It is the body, not the vocabulary, that sets the floor. For anyone who wants to run this model, Q4_K_M is the setting to use.

Artifacts. The merge, conversion, quantization and evaluation scripts are in the project repository at <https://github.com/streetsolider/div-translation>, including the GGUF evaluation harness behind every llama.cpp number in this paper. The quantized models are released at <https://huggingface.co/str33t/madlad400-3b-mt-en-dv-news-v1-GGUF>, and the bf16 LoRA adapter they derive from remains at <https://huggingface.co/str33t/madlad400-3b-mt-en-dv-news-v1>.

References

- [1] Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2212.09720.
- [2] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.07339.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.14314.
- [4] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.
- [5] Georgi Gerganov. k-quants. llama.cpp pull request #1684, <https://github.com/ggml-org/llama.cpp/pull/1684>, 2023.
- [6] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.
- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.
- [8] Sneha Kudugunta, Isaac Caswell, Biao Zhang, Xavier Garcia, Derrick Xin, Aditya Kusupati, Romi Stella, Ankur Bapna, and Orhan Firat. MADLAD-400: A multilingual and document-level large audited dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. arXiv:2309.04662.
- [9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2306.00978.
- [10] Maja Popović. chrF++: words helping character n-grams. In *Proceedings of the Second Conference on Machine Translation (WMT)*, pages 612–618, 2017.
- [11] Matt Post. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation (WMT)*, pages 186–191, 2018.
- [12] streetsolider. div-transliteration: the segha keymap for Thaana. <https://github.com/streetsolider/div-transliteration>, 2026.